

Account Validation Service (AVS) API: Low-Level Design

July 29, 2025

Prepared for: AVS API Development Team
Document Version: 1.0

Contents

1	Introduction	3
2	Low-Level Design Components	3
2.1	OpenAPI Specification	3
2.2	JSON Schemas	3
2.3	Error Handling	3
2.4	Security	4
2.5	Logging and Auditing	4
2.6	APIGEE Integration	4
2.7	External Service Integration	5
2.8	Performance and Scalability	5
2.9	Testing Strategy	5
2.10	Deprecation and Versioning	5
3	OpenAPI Specification	6
4	Example Request and Response	9
4.1	Request	9
4.2	Success Response	10
4.3	Error Response	10
5	Integration with Existing Design	10
6	Assumptions	10

1 Introduction

This document provides a detailed low-level design for the Account Validation Service (AVS) API, exposed via APIGEE. The design includes an OpenAPI 3.0 specification with JSON schemas for request and response payloads, error handling, security mechanisms, logging, and integration details. The API validates account details, including routing numbers and fraud checks, and supports versioning, auditing, and subscription management through APIGEE.

2 Low-Level Design Components

2.1 OpenAPI Specification

The OpenAPI 3.0 specification defines the API endpoints, request/response schemas, headers, error codes, and authentication mechanisms. It ensures clear documentation for clients, supports APIGEE's developer portal, and enables automated testing and client code generation.

- **Format:** OpenAPI 3.0, hosted in APIGEE's developer portal.
- **Details:** Includes JSON schemas, headers (`Company-ID`, `User-ID`), and OAuth 2.0/API key authentication.
- **Endpoints:** `/api/v1/validate` for account validation.

2.2 JSON Schemas

JSON schemas define the structure of request and response payloads, ensuring data consistency and validation.

- **Request Schema:** Specifies required fields like `routingNumber`, `accountNumber`, and `routingType`.
- **Response Schema:** Includes success (`status`, `routingNumberValid`, `fraudCheckResult`) and error (`errorCode`, `message`, `traceId`) formats.

2.3 Error Handling

Errors are standardized with specific codes, messages, and Trace IDs for traceability.

- **Error Codes:**
 - `INVALID_ENTITLEMENTS`: User or company lacks AVS access.
 - `INVALID_PAYLOAD`: Malformed JSON or missing fields.
 - `INVALID_ROUTING_NUMBER`: Routing number fails GUS validation.
 - `FRAUD_CHECK_FAILED`: Fraud Services API indicates high risk.
 - `INVALID_VERSION`: Unsupported API version.
- **Structure:** JSON with `errorCode`, `message`, `traceId`.

- **HTTP Status Codes:** 400 (client errors), 401 (unauthorized), 429 (rate limit), 500 (server errors).

2.4 Security

Security mechanisms protect sensitive data and ensure authorized access.

- **Authentication:** APIGEE enforces OAuth 2.0 (client credentials) or API keys (X-API-Key).
- **Entitlement Validation:** Validates `Company-ID` and `User-ID` via Entitlement Service, cached in Redis (5-minute TTL, if compliant).
- **Encryption:** HTTPS for requests, encrypt audit logs at rest.
- **Input Sanitization:** Validate `routingNumber` format (e.g., 9 digits for ABA).

2.5 Logging and Auditing

Audit logs capture request/response details for compliance and debugging.

- **Log Structure:**

```

1 {
2   "traceId": "123e4567-e89b-12d3-a456-426614174000",
3   "version": "v1",
4   "companyId": "COMP123",
5   "userId": "USER456",
6   "request": { /* Full request payload */ },
7   "response": { /* Full response or error */ },
8   "timestamp": "2025-07-29T21:52:00Z",
9   "status": "SUCCESS|ERROR"
10 }
```

- **Storage:** PostgreSQL with indexing on `traceId`.
- **APIGEE:** Log `traceId`, `Company-ID`, `User-ID` to analytics.

2.6 APIGEE Integration

APIGEE manages authentication, rate limiting, and subscription.

- **Proxy:** Routes `/api/v1/validate` to AVS backend.
- **Policies:**
 - Rate Limiting: 100 requests/minute/client.
 - Quota: 10,000 requests/day/subscription.
 - Spike Arrest: 10 requests/second.
 - Logging: Capture metadata in APIGEE analytics.
- **Developer Portal:** Host OpenAPI spec, subscription instructions, and deprecation notices.

2.7 External Service Integration

Interfaces for GUS and Fraud Services APIs ensure reliable validation.

- **GUS Service API:**
 - Endpoint: POST `/gus/validate-routing`.
 - Request: `{"routingNumber": "123456789", "type": "ABA"}`.
 - Response: `{"valid": true, "details": {}}`.
 - Retry: 3 attempts, exponential backoff (100ms, 200ms, 400ms).
 - Timeout: 500ms.
- **Fraud Services API:**
 - Endpoint: POST `/fraud/check`.
 - Request: `{"accountNumber": "9876543210", "routingNumber": "123456789"}`.
 - Response: `{"isFraudulent": false, "score": 10}`.
 - Retry: 2 attempts, 200ms backoff.
 - Timeout: 1 second.
- **Circuit Breaker:** Use `resilience4j` or `polly` to prevent cascading failures.

2.8 Performance and Scalability

The AVS API is designed for high throughput and scalability.

- **Horizontal Scaling:** Deploy on AWS ECS with auto-scaling.
- **Caching:** Redis for entitlement/routing number results (5-minute TTL, if compliant).
- **Database:** PostgreSQL with partitioning by date, indexed on `traceId`, `timestamp`.
- **APIGEE:** Load balancing across AVS instances.

2.9 Testing Strategy

Comprehensive testing ensures reliability.

- **Unit Tests:** Validate JSON schema, entitlement checks, error responses.
- **Integration Tests:** Mock GUS/Fraud APIs, test APIGEE proxy.
- **Load Tests:** Simulate 1000 requests/second with JMeter.
- **Version Tests:** Verify `/v1/validate` and `/v2/validate` behavior.

2.10 Deprecation and Versioning

Support for multiple versions ensures smooth transitions.

- **Version Support:** Maintain `/v1/` for 12 months after `/v2/` release.

- **Deprecation Notices:** Use `Deprecation: true` and `Sunset: 2026-07-10` headers.
- **Migration Guide:** Document changes in APIGEE portal.

3 OpenAPI Specification

The following OpenAPI 3.0 specification defines the `/api/v1/validate` endpoint, including JSON schemas, authentication, and error responses.

```

1 openapi: 3.0.3
2 info:
3   title: Account Validation Service (AVS) API
4   description: API for validating account details, including routing
   number and fraud checks, exposed via APIGEE.
5   version: 1.0.0
6 servers:
7   - url: https://api.example.com/api/v1
8     description: Version 1 API
9 components:
10  securitySchemes:
11    OAuth2:
12      type: oauth2
13      flows:
14        clientCredentials:
15          tokenUrl: https://api.example.com/oauth/token
16          scopes:
17            validate: Access to account validation endpoint
18    ApiKey:
19      type: apiKey
20      in: header
21      name: X-API-Key
22  schemas:
23    ValidationRequest:
24      type: object
25      required:
26        - routingNumber
27        - accountNumber
28        - routingType
29      properties:
30        routingNumber:
31          type: string
32          description: Routing number (ABA, SWIFT, or CHIPS).
33          example: "123456789"
34        accountNumber:
35          type: string
36          description: Account number to validate.
37          example: "9876543210"
38        routingType:
39          type: string
40          enum: [ABA, SWIFT, CHIPS]

```

```

41         description: Type of routing number.
42         example: ABA
43 ValidationResponse:
44     type: object
45     required:
46         - status
47         - routingNumberValid
48         - fraudCheckResult
49     properties:
50         status:
51             type: string
52             enum: [SUCCESS, ERROR]
53             description: Status of the validation request.
54             example: SUCCESS
55         routingNumberValid:
56             type: boolean
57             description: Indicates if the routing number is valid.
58             example: true
59         fraudCheckResult:
60             type: object
61             description: Result from Fraud Services API.
62             properties:
63                 isFraudulent:
64                     type: boolean
65                     description: Indicates if the account is flagged as
66                         fraudulent.
67                     example: false
68                 score:
69                     type: number
70                     description: Fraud risk score (0-100).
71                     example: 10
72         traceId:
73             type: string
74             description: Unique identifier for tracing the request (
75                 included in errors).
76             example: "123e4567-e89b-12d3-a456-426614174000"
77 ErrorResponse:
78     type: object
79     required:
80         - errorCode
81         - message
82         - traceId
83     properties:
84         errorCode:
85             type: string
86             description: Specific error code for the failure.
87             example: INVALID_ROUTING_NUMBER
88         message:
89             type: string
90             description: Human-readable error description.
91             example: Routing number is not valid for ABA format

```

```

90         traceId:
91             type: string
92             description: Unique identifier for tracing the request.
93             example: "123e4567-e89b-12d3-a456-426614174000"
94 headers:
95     Deprecation:
96         description: Indicates if the API version is deprecated.
97         schema:
98             type: string
99             example: true
100    Sunset:
101        description: Date when the API version will be decommissioned.
102        schema:
103            type: string
104            format: date
105            example: 2026-07-10
106 paths:
107     /validate:
108         post:
109             summary: Validate account details
110             description: Validates a routing number and account details,
111                 including entitlement checks and fraud detection.
112             operationId: validateAccount
113             security:
114                 - OAuth2: [validate]
115                 - ApiKey: []
116             parameters:
117                 - name: Company-ID
118                   in: header
119                   required: true
120                   schema:
121                       type: string
122                       description: Unique identifier for the company.
123                       example: "COMP123"
124                 - name: User-ID
125                   in: header
126                   required: true
127                   schema:
128                       type: string
129                       description: Unique identifier for the user.
130                       example: "USER456"
131             requestBody:
132                 required: true
133                 content:
134                     application/json:
135                         schema:
136                             $ref: '#/components/schemas/ValidationRequest'
137             responses:
138                 '200':
139                     description: Successful validation response
140                     content:

```



```

140         application/json:
141             schema:
142                 $ref: '#/components/schemas/ValidationResponse'
143     '400':
144         description: Client error (e.g., invalid payload, routing
145                     number, or entitlements)
146         content:
147             application/json:
148                 schema:
149                     $ref: '#/components/schemas/ErrorResponse'
150     '401':
151         description: Unauthorized access (invalid API key or OAuth
152                     token)
153         content:
154             application/json:
155                 schema:
156                     $ref: '#/components/schemas/ErrorResponse'
157     '429':
158         description: Rate limit exceeded
159         content:
160             application/json:
161                 schema:
162                     $ref: '#/components/schemas/ErrorResponse'
163     '500':
164         description: Server error
165         content:
166             application/json:
167                 schema:
168                     $ref: '#/components/schemas/ErrorResponse'
169     headers:
170         Deprecation:
171             $ref: '#/components/headers/Deprecation'
172         Sunset:
173             $ref: '#/components/headers/Sunset'

```

4 Example Request and Response

4.1 Request

```

1 POST /api/v1/validate HTTP/1.1
2 Host: api.example.com
3 Company-ID: COMP123
4 User-ID: USER456
5 Authorization: Bearer <token>
6 Content-Type: application/json
7
8 {
9     "routingNumber": "123456789",
10    "accountNumber": "9876543210",
11    "routingType": "ABA"

```

```
12 }
```

4.2 Success Response

```
1 HTTP/1.1 200 OK
2 Content-Type: application/json
3
4 {
5     "status": "SUCCESS",
6     "routingNumberValid": true,
7     "fraudCheckResult": {
8         "isFraudulent": false,
9         "score": 10
10    },
11     "traceId": "123e4567-e89b-12d3-a456-426614174000"
12 }
```

4.3 Error Response

```
1 HTTP/1.1 400 Bad Request
2 Content-Type: application/json
3
4 {
5     "errorCode": "INVALID_ROUTING_NUMBER",
6     "message": "Routing number is not valid for ABA format",
7     "traceId": "123e4567-e89b-12d3-a456-426614174000"
8 }
```

5 Integration with Existing Design

The design aligns with the high-level system and sequence diagrams:

- **System Diagram:** The AVS API Service handles `/v1/validate` requests, with APIGEE enforcing authentication and rate limiting. Audit logs are stored in a database.
- **Sequence Diagram:** Steps include entitlement validation, payload validation, GUS/Fraud API calls, and logging, all reflected in the OpenAPI specification.
- **Versioning:** Supports `/v1/validate`, with APIGEE routing for future `/v2/` versions.

6 Assumptions

- GUS and Fraud Services APIs have well-defined endpoints and respond in `<500ms`.
- APIGEE is configured with OAuth 2.0 or API key authentication.
- AVS API Service is stateless and deployed on a scalable platform (e.g., AWS).

- Compliance allows caching of entitlement/routing number results.